

Migração de dados relacionais para MongoDB

Loja de itens de League of Legends: UA 1 e 2

Curso: Pós-Graduação em Data Science e Analytics

Disciplina: Banco de Dados e Big Data para Data Science

Equipe: Diogo Galvão Carvalho; Felipe Artur Macedo Lima; Luan Cavalcante Dias Rodrigues

Data: Setembro de 2026

Repositório: https://github.com/FelipeArtur/bigdata_datascience

1. Introdução

Este trabalho implementa e verifica a migração de um banco PostgreSQL para MongoDB usando um cenário didático de compras de itens durante partidas de League of Legends. O objetivo é representar os mesmos dados em dois modelos, justificar o uso de documentos embutidos e referências e demonstrar que as transformações preservam o conteúdo transacional.

A modelagem parte de uma pergunta: como recuperar o perfil e as compras de um jogador sem reconstruir todas as relações a cada consulta, mantendo um catálogo compartilhado e evitando duplicar os dados completos das partidas? No modelo relacional, chaves estrangeiras representam os vínculos e o banco verifica sua existência. No modelo documental, parte dessas relações passa a ser representada pela estrutura do próprio documento; outras continuam como identificadores de documentos externos.

O MongoDB permite reunir perfil e compras em arrays e objetos aninhados, o que atende ao padrão de acesso escolhido. PostgreSQL também atende ao cenário, com suporte a transações e consultas com relacionamentos. A comparação trata das escolhas de representação; não mede latência nem estabelece que um modelo seja superior ao outro em qualquer situação.

A implementação usa Python para extração, validação, transformação e carga. O ambiente é reproduzível em Docker e contém PostgreSQL 16, MongoDB 7 e JupyterLab. A etapa seguinte utiliza a mesma base em Apache Spark e Delta Lake.

Adaptação de ambiente: o roteiro original solicita MongoDB Atlas. Por opção de execução integralmente local, este trabalho utiliza MongoDB em container. Não houve execução em Atlas nem medição de serviços de nuvem. A equivalência dessa adaptação para avaliação depende da aceitação do docente.

Natureza dos dados: o catálogo bruto de itens está versionado no repositório. O CSV identifica a versão 16.19.1 e a coleta em 25/09/2026 às 01:42:44.216 UTC, com URLs de origem do Data Dragon da Riot Games. A análise usa esse snapshot do catálogo, que pode diferir de versões posteriores. Perfis, partidas, compras e demografia são dados sintéticos. Nicks conhecidos são identificadores ilustrativos e não evidência de comportamento das pessoas citadas. Ouro é uma unidade do cenário, não faturamento monetário.

2. Cenário e modelo relacional

Cada jogador possui um elo e uma região. Durante uma partida, realiza compras de itens em determinadas quantidades e minutos. Um item pode ter várias categorias e cada categoria pode classificar vários itens. A tabela associativa resolve essa relação N:N.

Tabela	Chave primária	Vínculos	Linhas
elo	id_elo	Domínio de ranques	10
categoria	id_categoria	Domínio de categorias	32
item	item_id	Catálogo	254
item_categoria	item_id, id_categoria	item e categoria	834
jogador	id_jogador	id_elo	25
partida	id_partida	Entidade de partida	30
compra	id_compra	jogador, item e partida	1.043

Além das chaves da tabela, os atributos são: elo.elo; categoria.nome_categoria; item.nome, preco_unitario e descricao; jogador.nick e regioao; partida.data_partida, hora_partida, duracao_minutos e resultado; compra.data_compra, minuto_compra, quantidade, preco_unitario e total_compra. A associação item_categoria contém suas duas chaves.

```
elo (1) — (N) jogador (1) — (N) compra (N) — (1) partida
                        |
                        (N)
                        |
                        (1)
                        item
                        | (1)
                        | (N)
                    item_categoria
                        | (N)
                        | (1)
                        categoria
```

O DDL em docker/init-db/01-schema.sql contém PKs, FKs e restrições CHECK para valores e quantidades. total_compra é uma redundância deliberada, validada pela igualdade quantidade * preco_unitario. Essa redundância impede classificar todos os atributos como estritamente normalizados.

O gerador usa seed 42. Seleciona dez participantes por partida e produz de duas a cinco compras por participante. O minuto da compra permanece dentro da duração da partida; a data coincide com a partida. Seis itens com preço bruto zero recebem preço simbólico de 150 ouro, uma premissa da simulação. O resultado da partida é um rótulo global ilustrativo, não uma vitória atribuível a cada jogador, pois equipes não foram modeladas.

Os CSVs são carregados no PostgreSQL pelo inicializador Docker. A migração usada nesta execução extrai as sete tabelas desse banco, em uma transação de leitura com isolamento REPEATABLE READ. O modo CSV existe apenas como alternativa offline explícita.

3. Modelo documental e justificativas

Origem	Destino	Decisão
jogador, elo, compra	jogadores	Elo textual e compras embutidos; IDs preservados
item, item_categoria, categoria	itens	Categorias com ID e nome embutidas no catálogo
partida	partidas	Documento independente referenciado nas compras
elo, categoria completos	dominios	Um documento preserva todos os valores, inclusive os não utilizados

3.1 Embedding em jogadores

O histórico de compras pertence a um jogador e costuma ser consultado com seu perfil. Embuti-lo permite obter esses dados em uma única consulta. O campo `total_transacoes` tem o mesmo nome no script, no notebook e na coleção. Totais derivados de ouro e unidades são calculados a partir das compras, e os documentos são comparados integralmente após a carga.

Na amostra, o histórico cabe no documento do jogador. Ainda assim, sua leitura, armazenamento e manutenção têm custos. Arrays crescentes podem tornar documentos grandes e concentrar atualizações. MongoDB limita documentos BSON a 16 MiB. A seção de validação informa o maior tamanho observado nesta amostra; um histórico de anos exigiria outra medição. Em produção, seria necessário limitar o histórico embutido ou mover compras antigas para coleção própria.

3.2 Referências a itens e partidas

Compras conservam `id_item` e `id_partida`. Catálogo e partidas são compartilhados; copiar suas estruturas completas para cada compra aumentaria a redundância. Consultas que precisem de todos os atributos dessas entidades ainda exigem leituras adicionais ou agregação com `$lookup`.

O preço vem da compra original, preservando o valor transacional mesmo após alterações no catálogo. O nome do item é copiado do catálogo no momento da migração. Como a origem não guarda versões dos nomes, o nome copiado pode diferir daquele usado na data da compra.

Categorias são incorporadas ao item, conservando seus identificadores. A coleção `dominios` evita perder elos sem jogadores e mantém os domínios completos. A tabela associativa deixa de existir como coleção separada, mas seus vínculos continuam representados nos arrays.

3.3 Integridade e atomicidade

MongoDB não fornece FKs declarativas entre essas coleções. A aplicação verifica IDs únicos, referências, cálculo de totais e coerência temporal antes da carga. Validação `jsonSchema` poderia reforçar tipos e campos, mas não verificaria a existência de um documento referenciado em outra coleção.

A atomicidade de uma atualização documental não torna toda a migração atômica. O carregamento escreve coleções temporárias, confere documentos e só então substitui cada coleção de destino por `rename`. Uma falha durante a preparação preserva as coleções existentes. Uma falha entre renames pode deixar versões diferentes entre coleções; a execução pressupõe ausência de leitores concorrentes e permite nova execução corretiva.

3.4 Exemplos reais dos documentos

Os exemplos abaixo são produzidos a partir da mesma transformação utilizada na carga. O documento de jogador mostra somente a primeira compra; os totais correspondem ao histórico completo. Campos adicionais foram omitidos nos exemplos para facilitar a leitura.

jogadores (recorte):

```
{ "_id": 1, "nick": "Faker", "elo": "Desafiante", "total_transacoes": 37, "compras":  
  [{"id_compra": 72, "id_item": 3508, "id_partida": 3, "data_compra": "2025-08-21",  
    "minuto_compra": 18, "quantidade": 1, "preco_unitario": 3050, "item": "Colhedor de  
    Essência", "total_ouro": 3050}] }
```

itens (recorte):

```
{ "_id": 3508, "nome": "Colhedor de Essência", "preco_unitario": 3050, "categorias":  
  [{"id_categoria": 1, "nome_categoria": "AbilityHaste"}] }
```

partidas (recorte):

```
{ "_id": 3, "id_partida": 3, "data_partida": "2025-08-21", "hora_partida": "20:47:00",  
  "duracao_minutos": 35, "resultado": "Vitoria" }
```

dominios (recorte):

```
{ "_id": "dominios", "elos": [{"id_elo": 1, "elo": "Ferro"}] }
```

3.5 Escalabilidade, elasticidade e limites

Os documentos de jogadores poderiam ser distribuídos entre shards, mas o projeto não configura sharding nem valida uma chave de distribuição. Escolhê-la exigiria examinar distribuição, cardinalidade e consultas; uma chave com região não garante automaticamente localização geográfica dos dados.

Escalabilidade horizontal significa distribuir dados e trabalho; elasticidade envolve ajustar capacidade à demanda. Nenhuma delas foi medida neste ambiente local. No Atlas gratuito, sharding não está disponível. Crescimento real também exigiria políticas de histórico, índices, backups e capacidade, não apenas trocar o modelo lógico.

O esquema flexível permite acrescentar atributos. Essas alterações ainda exigem controle do contrato de dados. A aplicação e os consumidores ainda precisam concordar sobre campos, tipos e significado. O preço dessa flexibilidade é assumir validações que antes eram parcialmente declaradas no banco relacional.

4. Implementação e verificação

O notebook 02 importa a migração de `scripts/migrate_to_mongodb.py`. Assim, script e notebook executam o mesmo algoritmo.

```
tables = load_tables(source="postgres")
documents = build_documents(tables)
result = migrate(source="postgres")
```

A sequência executada foi: extração relacional consistente; validação de todas as tabelas; construção documental; inserção em coleções temporárias; comparação completa dos documentos lidos; criação do índice de região; substituição das coleções; gravação de evidência sem URI ou senha. Erros de uma carga solicitada retornam código diferente de zero.

Verificação	Resultado
Origem efetiva	postgres
Destino	MongoDB local
Data da execução (UTC)	2026-09-29T00:05:02.650134+00:00
Coleções e documentos	itens: 254, partidas: 30, jogadores: 25, dominios: 1
Compras / ouro	1043 / 2228821
Comparação após carga	Igualdade dos documentos completos
Maior documento de jogador (BSON)	11170 bytes
Índices em jogadores	<code>id</code> , <code>regiao_1</code>

Exemplos de consultas presentes no notebook executado:

```
db.jogadores.find_one({"_id": 1}, {"compras": {"$slice": 1}})
db.jogadores.aggregate([
  {"$group": {"_id": "$regiao",
               "ouro": {"$sum": "$total_gasto_ouro"}}},
  {"$sort": {"ouro": -1}}
])
```

A migração cria o índice `regiao_1` e verifica sua existência. Em uma coleção pequena, o otimizador pode preferir uma varredura ao uso do índice. Como o projeto não cronometra consultas equivalentes em PostgreSQL e MongoDB, não é possível concluir que a migração reduziu a latência.

Os testes de regressão incluem referência órfã rejeitada antes da exportação e falha de carga com retorno não zero. A continuidade é verificada na Entrega 2: todas as compras extraídas pelo Spark são confrontadas com o PostgreSQL, e a saída Delta é relida e comparada.

5. Conclusão e avaliação crítica

A comparação após a carga confirmou que a migração preservou o conteúdo transacional. O documento do jogador reúne perfil e compras para atender à consulta prevista. As referências preservam entidades compartilhadas; os arrays de categorias representam a relação N:N sem uma coleção associativa adicional.

O validador rejeita inconsistências antes da carga. Depois da inserção, a comparação verifica o conteúdo completo, além das contagens. As coleções temporárias preservam a base existente caso a preparação falhe, mas a substituição das quatro coleções continua sujeita a falhas entre as trocas.

A simulação é pequena e não permite concluir que MongoDB seja mais rápido ou mais escalável do que PostgreSQL neste domínio. Também não sustenta inferências sobre jogadores reais. A execução verificou a transformação e a conservação dos dados em um ambiente local reproduzível. Elasticidade e sharding foram discutidos conceitualmente, sem serem apresentados como resultados experimentais.

Se o projeto crescer, será preciso avaliar o limite do histórico embutido, o registro de versões do catálogo, o desempenho de consultas equivalentes sob carga e a publicação consistente para leitores concorrentes. Essas medidas só devem ser implementadas se o volume e os requisitos as justificarem.

A base resultante é consumida pelo pipeline Spark da Entrega 2. Cabe ao docente aceitar a execução local como substituição do Atlas solicitado no roteiro.

Referências

MONGODB, INC. **Data modeling**. Disponível em: <https://www.mongodb.com/docs/manual/data-modeling/>. Acesso em: 28 set. 2026.

MONGODB, INC. **Schema validation**. Disponível em: <https://www.mongodb.com/docs/manual/core/schema-validation/>. Acesso em: 28 set. 2026.

MONGODB, INC. **MongoDB limits and thresholds**. Disponível em: <https://www.mongodb.com/docs/manual/reference/limits/>. Acesso em: 28 set. 2026.

MONGODB, INC. **Atlas free cluster limits**. Disponível em: <https://www.mongodb.com/docs/atlas/reference/free-shared-limitations/>. Acesso em: 28 set. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **Constraints**. PostgreSQL 16 Documentation. Disponível em: <https://www.postgresql.org/docs/16/ddl-constraints.html>. Acesso em: 28 set. 2026.